

# Victor Mendonça

Principal Software Engineer · Platform Architect · AI Systems, Data Platforms & Multi-Tenant SaaS

São Paulo, Brazil · [github.com/mend3](https://github.com/mend3) · [linkedin.com/in/vicmendonca](https://linkedin.com/in/vicmendonca) · [victor.mendonca@live.com](mailto:victor.mendonca@live.com)

## EXECUTIVE SUMMARY

Software engineer, platform architect, and technical leader with **15+ years** designing, building, and operating the platforms that companies run on — multi-tenant SaaS, distributed systems, large-scale data acquisition, and AI-native products.

One thread connects the work across **fintech, legaltech, retail & commerce intelligence, and data-intensive platforms**: taking fragmented data, integrations, and processes and turning them into **operational intelligence** — systems a business operates on, not dashboards it looks at. I did it consolidating millions of pricing signals for global brands; I'm doing it now in DevShell One, where many products share one foundation.

I operate across the full arc — **set the architecture and strategy, then write the code that de-risks the hardest parts** — and I treat structural decisions as business decisions: every one tied to cost, reliability, or speed-to-market.

## CORE PLATFORM DOMAINS

|                           |                                                                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Multi-tenant SaaS         | Most SaaS products rebuild the same things — auth, billing, permissions, integrations. I treat those as platform capabilities instead of re-solving them per product. |
| Data acquisition at scale | Reliable supply of external data when the web fights back — distributed crawling, browser automation, proxy management and anti-bot.                                  |
| Integration ecosystems    | Many systems made to behave as one, on a single canonical data model — no more silos that disagree.                                                                   |
| AI & agentic systems      | Most AI stalls because context is scattered. The work is the infrastructure that lets a model reason over products, users, documents and operational data at once.    |
| Operational intelligence  | Fragmented signals turned into what to do next — pricing, shelf, performance, growth.                                                                                 |
| Reliability & cost        | A system that needs tribal knowledge to run isn't finished — observable, resilient, and cheaper to operate over time.                                                 |

## PROFESSIONAL EXPERIENCE

---

### Founder & Platform Architect — *DevShell One*

2023 – Present

**Built:** a single platform where one shared foundation powers many products and verticals — instead of rebuilding infrastructure, data, and AI for each.

**Problem:** every new product or vertical normally means re-creating infrastructure, integrations, and AI from scratch.

**Impact:** verticals now ship as modules on a shared base, so the marginal cost of the next product, tenant, or market keeps falling — and the AI understands the whole operation because everything lives on one data model.

- **Shared control plane** — one foundation every product runs on: orchestration, shared databases, AI infrastructure, vector search, and observability.
- **Multi-tenant application runtime** — the layer where a whole vertical becomes a module: universal connectors, content/CMS and page building, workflow automation, scheduling, recruitment, real estate, and dashboards. Many products, one codebase.
- **Data acquisition layer** — resilient, large-scale collection (distributed crawling, browser automation, proxy management, anti-bot) that supplies the rest of the platform.
- **Native AI layer** — an assistant that understands the whole system: persistent memory, knowledge retrieval, agentic workflows, and tool execution, with data and inference able to stay on the company's own infrastructure.

Ownership across product strategy, software architecture, infrastructure, AI systems, data platforms, and engineering standards.

*TypeScript · Node.js · Next.js · React · PostgreSQL · Redis · Kubernetes · local LLMs (Ollama) · vector search · event-driven architecture*

### Senior Software Engineer — *Visualitics*

2025 – Present

**Built:** the integration and processing core of a commerce-intelligence platform that consolidates marketplace, advertising, and analytics data into a single decision layer — surfaced in-workflow through web apps and a browser extension.

**Problem:** a seller's performance data is scattered across a dozen marketplaces and ad networks that never agree.

**Impact:** one place to see and act on the whole operation, with insight delivered where the work already happens.

Owned multi-source ingestion, distributed processing, ETL, and KPI computation across Mercado Livre, Shopee, Amazon SP-API, VTEX, NuvemShop, GA4, Meta Ads, and Google Ads.

*Node.js · TypeScript · Express · Prisma · Python · FastAPI · BigQuery · Redis · GCP · browser-extension architecture*

### Senior Software Engineer — *Jusbrasil*

2024 – 2025

**Built:** the large-scale web-access and data-acquisition infrastructure behind the company's intelligence operations — proxy and routing architecture for fast, dependable retrieval at scale.

**Problem:** reliable data acquisition gets expensive and brittle as volume and anti-bot pressure grow.

**Impact:** materially lower operational spend with stronger reliability — and internal tooling that lifted the whole team's throughput.

Reliability, resiliency, observability, and developer-experience work as first-class concerns.

*Node.js · TypeScript · Docker · Kubernetes · Redis · PostgreSQL · observability platforms*

## Senior Software Engineer — *Stone*

2023 – 2024

**Built:** backend services and integrations for high-volume payment processing and merchant acquiring at one of Latin America's largest acquirers.

**Problem:** money movement tolerates no downtime, no data loss, and no compliance gaps — at scale.

**Impact:** dependable, low-latency financial flows that hold under strict reliability and regulatory constraints.

Distributed systems, payment and banking integrations, and performance optimization under fintech-grade requirements.

*Node.js · TypeScript · distributed systems · cloud infrastructure*

## Full Stack Developer → Technical Lead → Technology Manager — *Intellibrand · Ascential*

2019 – 2023

**Built & led:** the retail- and commerce-intelligence platforms that turned **millions of pricing, inventory, and merchandising signals** from thousands of digital shelves into decisions global consumer brands could act on — and grew from writing that system to leading the team behind it.

**Problem:** global brands couldn't see how they were really performing across thousands of digital shelves, in too many channels to track by hand.

**Impact:** real-time digital-shelf, pricing and content intelligence used by global brands across Latin America and beyond — and a growing engineering team and the standards that held as data volumes climbed. This is where the through-line started: fragmented data → operational intelligence, at scale.

- **Full Stack Developer (2019–2021)** — built the ingestion pipelines, APIs and dashboards behind real-time retail analytics, across relational and NoSQL data.
- **Technical Lead (2021–2023)** — owned the scalable architecture for large-scale collection and analytics, pricing intelligence and digital-shelf insight; mentored engineers and set the delivery standards.
- **Technology Manager · Ascential (2023)** — led engineering across retail analytics, content & compliance and store-locator tooling; aligned engineering, product and data science, and owned the technical roadmap for global brands.

Where the through-line started — and where I grew from IC to manager on the same platform.

*Node.js · TypeScript · data pipelines · cloud · AI/ML · CI/CD*

## Full Stack Developer — *Zoroastro C. Teixeira Advogados*

2018 – 2019

Legal-operations software — case and deadline management, financial controls and bank reconciliation, and internal task automation (Node.js, Angular, MongoDB, AWS).

## Java Full Stack Developer — *Contalex*

2014 – 2017

A SaaS ERP for accounting firms — financial routines, recurring billing via a payments API, and customer onboarding; my first real taste of a multi-tenant SaaS product (Java, Spring, SQL Server, AngularJS, Azure).

## Web & Software Developer — *Early career*

2011 – 2014

Early career across software houses and retail — PHP web systems and ERPs, a desktop/RIA app, and e-commerce/mobile (CS-Consoft, RSWEBSITES, BLM, Orgamec). Where the fundamentals came from.

## SELECTED PLATFORM ACHIEVEMENTS

---

- **I designed and operate DevShell One end-to-end** — control plane, data acquisition, integration framework, workflow automation, AI layer, and vertical apps — proving one foundation can power many products at a falling marginal cost.
- **I built retail & commerce-intelligence platforms processing millions of pricing, inventory, and merchandising signals** across global marketplaces, used by global consumer brands to steer pricing, availability, and digital-shelf strategy.
- **I architected large-scale, cost-efficient web-access and data-acquisition infrastructure** with reliability and observability as first-class concerns — cutting operational spend without sacrificing quality.

- **I delivered high-volume payments and financial systems** under strict reliability and compliance constraints, at one of Latin America's largest acquirers.
- **The consistent thread across fintech, legaltech, retail intelligence, and data platforms:** turning fragmented data, integrations, and processes into operational intelligence.

## TECHNICAL LEADERSHIP

---

- **Full-arc ownership** — set architecture and technical strategy, then write the code that de-risks the hardest parts. I don't hand off the unknown.
- **Force-multiplier engineering** — establish the standards, review culture, and platform conventions that let a small team ship like a much larger one.
- **Grew people and direction** — mentored engineers and owned roadmaps and technical decisions as Tech Lead and Technology Manager.
- **Architecture as a business lever** — every structural decision tied to a concrete outcome: lower cost, higher reliability, or faster time-to-market.

## ARCHITECTURE EXPERTISE

---

- **Platforms** — multi-tenant SaaS · distributed & event-driven systems · domain-driven design · system design
- **Data & integration** — connector frameworks · canonical data modeling · ETL & analytical pipelines · large-scale acquisition
- **AI-native** — agentic systems · retrieval-augmented generation · knowledge systems · vector search · local-first inference
- **Operations** — reliability & observability · cost engineering · infrastructure as a shared control plane

## TECHNOLOGIES & TOOLS

---

*Tools in service of the systems above — not the other way around.*

- **Architecture & Leadership** — Platform Architecture · Software Architecture · Distributed Systems · Multi-Tenant SaaS · Systems Design · Solution Architecture · Engineering Strategy · Technical Leadership
- **AI & Data** — LLMs · Agentic Systems · RAG · Knowledge Systems · Data Engineering · Data Platforms · ETL · Vector Databases
- **Backend** — TypeScript · Node.js · Express · NestJS · Java · Spring · REST APIs · Microservices
- **Frontend** — React · Next.js · Angular
- **Databases** — PostgreSQL · Redis · MongoDB · MySQL · ClickHouse · Qdrant · pgvector · BigQuery
- **Infrastructure** — Docker · Kubernetes · AWS · GCP · CI/CD · GitHub Actions · Linux · Observability

## ADDITIONAL INFORMATION

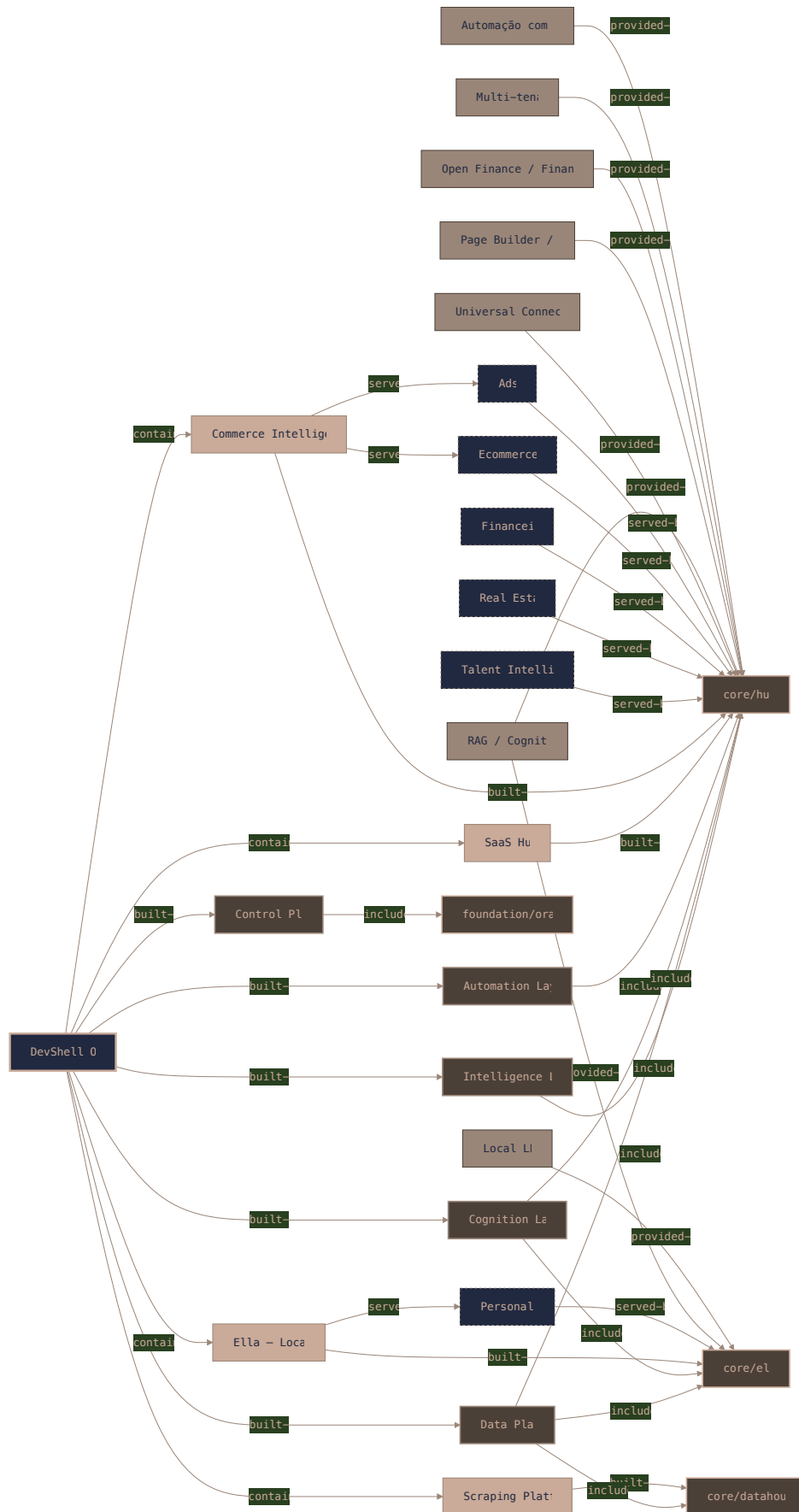
---

**Target roles:** Principal / Staff Software Engineer · Platform / Software / Solutions Architect · AI Systems Architect · Data Platform Architect · Engineering Manager · Head / Director of Engineering · Fractional CTO

**Languages:** Portuguese (native) · English (professional)

**Location:** São Paulo, Brazil

## PLATFORM ARCHITECTURE – ECOSYSTEM MAP



Mapped from the real running system I architect and operate — vision → products → domains → capabilities → projects.